Module 3

1. Explore Chucking process to extract Noun phrase

2. Explore various Language corpora

3. Analyse words with WordNet with different operations

1. Explore Chucking process to extract Noun phrase

- Chunking requires POS tags
- Use POS tags to extract phrases

Step 1: Use Preprocessed Text

Take:

- Cleaned text (from Module 1)
- Tokens (from Module 1)
- POS Tagged output (from Module 2)

Example from Module 2:

Subjective/NN
yearold/JJ
white/JJ
female/NN
presents/NNS

Step 2: Apply Chunk Grammar

Define Noun Phrase Rule:

NP → (DT)? (JJ)* (NN)+

Meaning:

- Optional determiner
- Zero or more adjectives
- One or more nouns

Step 3: Perform Chunking

Use the POS-tagged words as input.

The chunker groups:

natural language processing

as one Noun Phrase.

Code:

```
chunk_results = []
grammar = "NP: {<DT>?<JJ>*<NN>+}" # Step 2: Define Grammar
cp = nltk.RegexpParser(grammar)

for sent in sentences:
    clean_sent = re.sub(r'[^a-zA-Z\s]', '', sent.lower())
    tokens = word_tokenize(clean_sent)
    pos_tags = nltk.pos_tag(tokens)

    # Step 3: Perform Chunking
    chunked_tree = cp.parse(pos_tags)

    # Step 4: Extract Noun Phrases
    nps = []
    for subtree in chunked_tree.subtrees(filter=lambda t: t.label() == 'NP'):
        phrase = " ".join([word for word, tag in subtree.leaves()])
        nps.append(phrase)
```

Step 4: Extract Noun Phrases

Extract only subtrees labeled **NP**.

Expected Output

```
--- SECTION 1: NOUN PHRASE CHUNKING ---
```

	Sentence	Extracted Noun Phrase
0	SUBJECTIVE:, This 23-year-old white female pr...	subjective, this yearold white female, complaint
1	She used to have allergies when she lived in S...	seattle
2	In the past, she has tried Claritin, and Zyrte...	claritin, zyrtec
3	Both worked for short time but then seemed to ...	short time, effectiveness
4	She has used Allegra also....	

Chunking helps:

- Identify key information units
- Improve summarization
- Extract domain-specific phrases

2. Explore Various Language Corpora

Step 1: Select a Corpus

Use standard corpora like:

- Gutenberg Corpus

- Brown Corpus
- Reuters Corpus

Step 2: Load Corpus Data

Access:

- File IDs
- Categories
- Word lists

Step 3: Perform Basic Analysis

Apply techniques learned earlier:

- Token count
- Vocabulary size
- Frequency distribution
- Most common words

Step 4: Compare Categories

Example :

- News
- Editorial
- Fiction

Compare:

- Word usage
- Vocabulary richness

Category	Word Usage
News	formal words
Fiction	creative words
Editorial	opinion words

Code:

```
def analyze_words(word_list):
    alpha_words = [w.lower() for w in word_list if w.isalpha()]
```

```
    return [len(alpha_words), len(set(alpha_words)),
Counter(alpha_words).most_common(1)[0][0] if alpha_words else "N/A"]

all_mt_text = " ".join(df_mt['transcription'].dropna().astype(str))
results = [
    ["MTSamples (Medical)"] +
analyze_words(word_tokenize(all_mt_text[:100000])), # Sample of dataset for
speed
    ["Brown (News)"] + analyze_words(brown.words(categories='news')),
    ["Gutenberg"] + analyze_words(gutenberg.words(gutenberg.fileids()[0])),
    ["Reuters"] + analyze_words(reuters.words(reuters.fileids()[0]))
]

comparison_df = pd.DataFrame(results, columns=["Corpus", "Total Words",
"Vocabulary Size", "Most Frequent Word"])
display(comparison_df)
```

Output:

```
--- SECTION 2: CORPORA EXPLORATION ---
```

	Corpus	Total Words	Vocabulary Size	Most Frequent Word
0	MTSamples (Medical)	14492	2540	the
1	Brown (News)	83562	11151	the
2	Gutenberg	161600	7079	to
3	Reuters	751	347	the

```
Brown Corpus Category Comparison (Vocabulary Richness):
```

	Category	Tokens Per Unique Word
0	News	6.985827
1	Editorial	6.228918
2	Fiction	7.362718

Corpora help:

- Build language models
- Study real-world text patterns
- Train NLP systems

3. Analyse Words using WordNet

- Module 2 handled word structure (stem/lemma)
- Module 3 handles word meaning (semantics)

Resource Used

WordNet

Step-by-Step Process

Step 1: Select a Word

Example:

treatment

Step 2: Get Synsets

Retrieve all meanings of the word.

Step 3: Extract Synonyms

Example:

treatment→ intervention, discourse

Step 4: Find Antonyms

Example:

treatment→ None

Step 5: Hypernyms

treatment→ care

Step 6: Hyponyms

treatment→ orthoptics

Step 7: Meronyms

treatment→ remedy

Step 8: Compute Similarity

Compare:

treatment vs disease

Similarity score: 0.062

Code:

```
print("--- SECTION 3: MULTI-WORD WORDNET ANALYSIS ---")
```

```python
sample_words = ['treatment', 'doctor', 'heart', 'fever', 'surgery', 'patient',
'medicine']

def get_wordnet_data(word, compare_synset=None):
    syns = wn.synsets(word)
    if not syns:
        return None

    synonyms = set([l.name() for s in syns for l in s.lemmas()])
    syn_str = ", ".join(list(synonyms)[:2])

    ant = "None"
    if syns[0].lemmas()[0].antonyms():
        ant = syns[0].lemmas()[0].antonyms()[0].name()

    hyper = syns[0].hypernyms()[0].name().split('.')[0] if syns[0].hypernyms()
else "N/A"

    hypo = syns[0].hyponyms()[0].name().split('.')[0] if syns[0].hyponyms()
else "N/A"

    mero = syns[0].part_meronyms()[0].name().split('.')[0] if
syns[0].part_meronyms() else "N/A"

    sim = "N/A"
    if compare_synset and syns:
        val = syns[0].path_similarity(compare_synset)
        if val:
            sim = round(val, 3)

    return [word, syn_str, ant, hyper, hypo, mero, sim]

results_multi = []
base_word = 'disease'
base_synset = wn.synsets(base_word)[0] if wn.synsets(base_word) else None
for w in sample_words:
    data = get_wordnet_data(w, base_synset)
    if data:
        results_multi.append(data)

cols = ["Word", "Synonyms", "Antonym", "Hypernym", "Hyponym", "Meronym",
f"Similarity (to {base_word})"]
display(pd.DataFrame(results_multi, columns=cols))
print(f"\n*Similarity is calculated based on path similarity to
'{base_word}'.")
```

Output Table

```
--- SECTION 3: MULTI-WORD WORDNET ANALYSIS ---
```

	Word	Synonyms	Antonym	Hypernym	Hyponym	Meronym	Similarity (to disease)
0	treatment	discussion, treatment	None	care	chiropractic	remedy	0.062
1	doctor	Dr., furbish_up	None	medical_practitioner	specialist	N/A	0.056
2	heart	spirit, inwardness	None	intuition	N/A	N/A	0.062
3	fever	fever, feverishness	None	symptom	hyperpyrexia	N/A	0.067
4	surgery	operating_theatre, operating_room	None	medical_science	N/A	N/A	0.053
5	patient	patient, patient_role	None	sick_person	analysand	N/A	0.067
6	medicine	medicament, practice_of_medicine	None	medical_science	angiology	N/A	0.053

```
*Similarity is calculated based on path similarity to 'disease'.
```

Combined Outcome (1,2,3)

1. Chunking → Extracts meaningful noun phrases from sentences.
2. Corpora → Provides statistical analysis of real-world text data.
3. WordNet → Gives semantic relationships like synonym, antonym, hypernym, etc.

Result:

Thus, syntactic analysis (phrases), statistical text analysis (corpora), and semantic understanding using WordNet have been successfully implemented.